Inventory & Warehouse Management System

Presented by

Ankita Biswas [UG/02//BTCSECSF/2023/019]
Ankita Das [UG/02/BTCSECSF/2023/017]
Digant Mishra [UG/02/BTCSECSF/2023/016]
Saubantik Das [UG/02/BTCSECSF/2023/015]

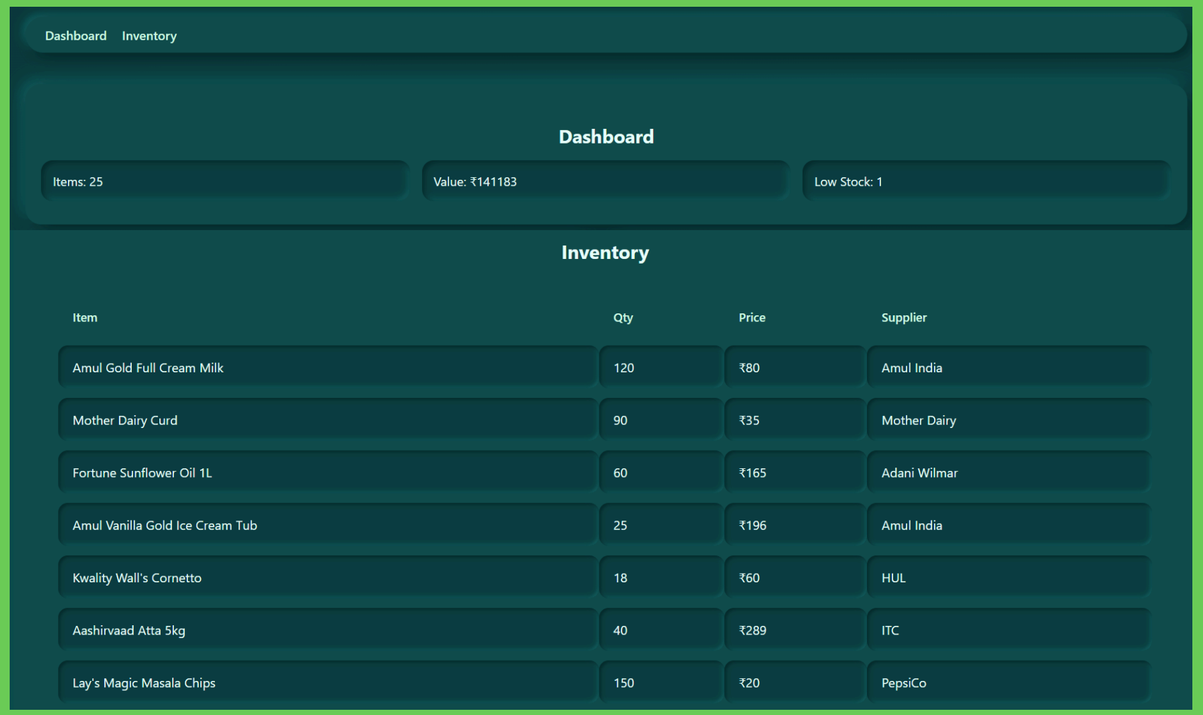
Unstructured Database | 2025-2026

Problem Statement

- Manual warehouse tracking often leads to stock mismatch and delayed replenishment
- Large product categories require faster search, update, and movement history
- A digital inventory system improves visibility, speed, and reporting accuracy

ABOUT

- Designed to manage stock availability and product movement in a warehouse environment
- Focused on real-time inventory tracking using a Swiggy Instamart-inspired use case
- Combines backend operations, database management, and dashboard visualization



Item	Qty	Price	Supplier
Amul Gold Full Cream Milk	120	₹80	Amul India
Mother Dairy Curd	90	₹35	Mother Dairy
Fortune Sunflower Oil 1L	60	₹165	Adani Wilmar
Amul Vanilla Gold Ice Cream Tub	25	₹196	Amul India
Kwality Wall's Cornetto	18	₹60	HUL
Aashirvaad Atta 5kg	40	₹289	ITC
Lay's Magic Masala Chips	150	₹20	PepsiCo

Database Design

Three MongoDB collections were used: items, suppliers, and transactions
The document-based model allows flexible product and stock storage
The structure supports easy CRUD, reporting, and movement tracking

```
>_MONGOSH
> use zwiggyInventoryDB
< switched to db zwiggyInventoryDB
> db.createCollection("items")
< { ok: 1 }
> db.createCollection("suppliers")
< { ok: 1 }
> db.createCollection("transaction")
< { ok: 1 }
```

Collection Creation and Data Insertion

- The database and collections were created using Mongo Shell
- Twenty-five realistic inventory products were inserted as sample warehouse stock
- The dataset was modeled around common Swiggy Instamart grocery categories.

```
> db.items.insertMany([
  {
    "item_id": 101,
    "name": "Amul Gold Full Cream Milk",
    "category": "Dairy",
    "brand": "Amul",
    "price": 80,
    "quantity": 120,
    "supplier": "Amul India"
  },
  {
    "item_id": 102,
    "name": "Mother Dairy Curd",
    "category": "Dairy",
    "brand": "Mother Dairy",
    "price": 35,
    "quantity": 90,
    "supplier": "Mother Dairy"
  },
```

Inserting the products in "items" collection


```
    "item_id": 103,
    "name": "Fortune Sunflower Oil 1L",
    "category": "Oil",
    "brand": "Fortune",
    "price": 165,
    "quantity": 60,
    "supplier": "Adani Wilmar"
  },
  {
    "item_id": 104,
    "name": "Amul Vanilla Gold Ice Cream Tub",
    "category": "Frozen",
    "brand": "Amul",
    "price": 196,
    "quantity": 25,
    "supplier": "Amul India"
  },
  {
    "item_id": 105,
    "name": "Kwality Wall's Cornetto",
    "category": "Frozen",
    "brand": "Kwality Wall's",
    "price": 60,
    "quantity": 18,
    "supplier": "HUL"
```

```
    "item_id": 106,
    "name": "Aashirvaad Atta 5kg",
    "category": "Staples",
    "brand": "Aashirvaad",
    "price": 289,
    "quantity": 40,
    "supplier": "ITC"
  },
  {
    "item_id": 107,
    "name": "Lay's Magic Masala Chips",
    "category": "Snacks",
    "brand": "Lay's",
    "price": 20,
    "quantity": 150,
    "supplier": "PepsiCo"
  },
  {
    "item_id": 108,
    "name": "Tata Sampann Toor Dal 1kg",
    "category": "Staples",
    "brand": "Tata Sampann",
    "price": 145,
    "quantity": 55,
    "supplier": "Tata Consumer"
  },
  {
```

```
    "item_id": 109,
    "name": "Maggi 2-Minute Noodles",
    "category": "Instant Food",
    "brand": "Maggi",
    "price": 14,
    "quantity": 200,
    "supplier": "Nestle"
  },
  {
    "item_id": 110,
    "name": "Britannia Good Day Biscuits",
    "category": "Biscuits",
    "brand": "Britannia",
    "price": 35,
    "quantity": 100,
    "supplier": "Britannia"
  },
  {
    "item_id": 111,
    "name": "Parle-G Biscuit",
    "category": "Biscuits",
    "brand": "Parle",
    "price": 10,
    "quantity": 250,
    "supplier": "Parle"
  },
```

Viewing the inserted data in the db

- Stored inventory data was retrieved using find queries
- Filtered searches were performed using category, quantity, and item_id
- This helped simulate real warehouse product lookup scenarios

```
> db.items.find()
< {
    _id: ObjectId('69cf96e7c697f6c3da26d7b3'),
    item_id: 101,
    name: 'Amul Gold Full Cream Milk',
    category: 'Dairy',
    brand: 'Amul',
    price: 80,
    quantity: 120,
    supplier: 'Amul India'
  }
  {
    _id: ObjectId('69cf96e7c697f6c3da26d7b4'),
    item_id: 102,
    name: 'Mother Dairy Curd',
    category: 'Dairy',
    brand: 'Mother Dairy',
    price: 35,
    quantity: 90,
    supplier: 'Mother Dairy'
  }
  {
    _id: ObjectId('69cf96e7c697f6c3da26d7b5'),
    item_id: 103,
    name: 'Fortune Sunflower Oil 1L',
    category: 'Oil',
    brand: 'Fortune',
    price: 165,
    quantity: 60,
```

Reading data in the "items" collection with different conditions

```
> db.items.find({ name: "Amul Gold Full Cream Milk" })
< {
    _id: ObjectId('69cf96e7c697f6c3da26d7b3'),
    item_id: 101,
    name: 'Amul Gold Full Cream Milk',
    category: 'Dairy',
    brand: 'Amul',
    price: 80,
    quantity: 120,
    supplier: 'Amul India'
  }
> db.items.find({ name: "Amul Gold Full Cream Milk" })
< {
    _id: ObjectId('69cf96e7c697f6c3da26d7b3'),
    item_id: 101,
    name: 'Amul Gold Full Cream Milk',
    category: 'Dairy',
    brand: 'Amul',
    price: 80,
    quantity: 120,
    supplier: 'Amul India'
  }
> db.items.find({ category: "Dairy" })
< {
    _id: ObjectId('69cf96e7c697f6c3da26d7b3'),
    item_id: 101,
    name: 'Amul Gold Full Cream Milk',
    category: 'Dairy',
    brand: 'Amul',
```

```
> db.items.find({ category: "Dairy" })
< {
    _id: ObjectId('69cf96e7c697f6c3da26d7b3'),
    item_id: 101,
    name: 'Amul Gold Full Cream Milk',
    category: 'Dairy',
    brand: 'Amul',
    price: 80,
    quantity: 120,
    supplier: 'Amul India'
  }
  {
    _id: ObjectId('69cf96e7c697f6c3da26d7b4'),
    item_id: 102,
    name: 'Mother Dairy Curd',
    category: 'Dairy',
    brand: 'Mother Dairy',
    price: 35,
    quantity: 90,
    supplier: 'Mother Dairy'
  }
```

```
> db.items.find({ quantity: { $lt: 30 } })
< {
    _id: ObjectId('69cf96e7c697f6c3da26d7b6'),
    item_id: 104,
    name: 'Amul Vanilla Gold Ice Cream Tub',
    category: 'Frozen',
    brand: 'Amul',
    price: 196,
    quantity: 25,
    supplier: 'Amul India'
  }
  {
    _id: ObjectId('69cf96e7c697f6c3da26d7b7'),
    item_id: 105,
    name: "Kwality Wall's Cornetto",
    category: 'Frozen',
    brand: "Kwality Wall's",
    price: 60,
    quantity: 18,
    supplier: 'HUL'
  }
```

Single Record Update

- Specific product details were updated using updateOne
- This was used for stock refill and price correction use cases
- The updated values were verified using retrieval queries

```
> db.items.updateOne(
    { item_id: 101 },
    { $set: { price: 85 } }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

Bulk Update

- updateMany was used for category-level stock updates

```
> db.items.updateMany(
    { category: "Dairy" },
    { $inc: { quantity: 20 } }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 2,
    modifiedCount: 2,
    upsertedCount: 0
  }
```

Record Deletion

- deleteOne and deleteMany handled discontinued and expired stock cleanup
- This reflects real warehouse inventory maintenance operations

```
> db.items.deleteMany({ category: "Breakfast" })
< {
    acknowledged: true,
    deletedCount: 0
  }
> db.items.deleteMany({ quantity: { $lt: 20 } })
< {
    acknowledged: true,
    deletedCount: 0
  }
```

Single Field Indexing

- A single-field index was created on item_id
- This improved transaction lookup for specific products
- Used for faster SKU-based stock movement searches

```
> db.transactions.createIndex({ item_id: 1 })
< item_id_1
```

Compound Indexing

- A compound index was created on item_id and date
- This optimized product movement queries over time
- Especially useful for stock history and audit-based searches

```
> db.transactions.createIndex({ item_id: 1, date: 1 })
< item_id_1_date_1
```

Aggregation for Inventory Valuation

- Total stock value was calculated using price and quantity multiplication
- This gives the financial worth of the current warehouse inventory
- Useful for inventory audits and stock investment analysis

```
> db.items.aggregate([
    {
      $group: {
        _id: null,
        totalStockValue: {
          $sum: { $multiply: ["$price", "$quantity"] }
        }
      }
    }
  ])
< {
    _id: null,
    totalStockValue: 130193
  }
> db.items.aggregate([
    {
      $group: {
        _id: null,
        totalStockValue: {
          $sum: { $multiply: ["$price", "$quantity"] }
        }
      }
    }
  ])
< {
    _id: null,
    totalStockValue: 130193
  }
```

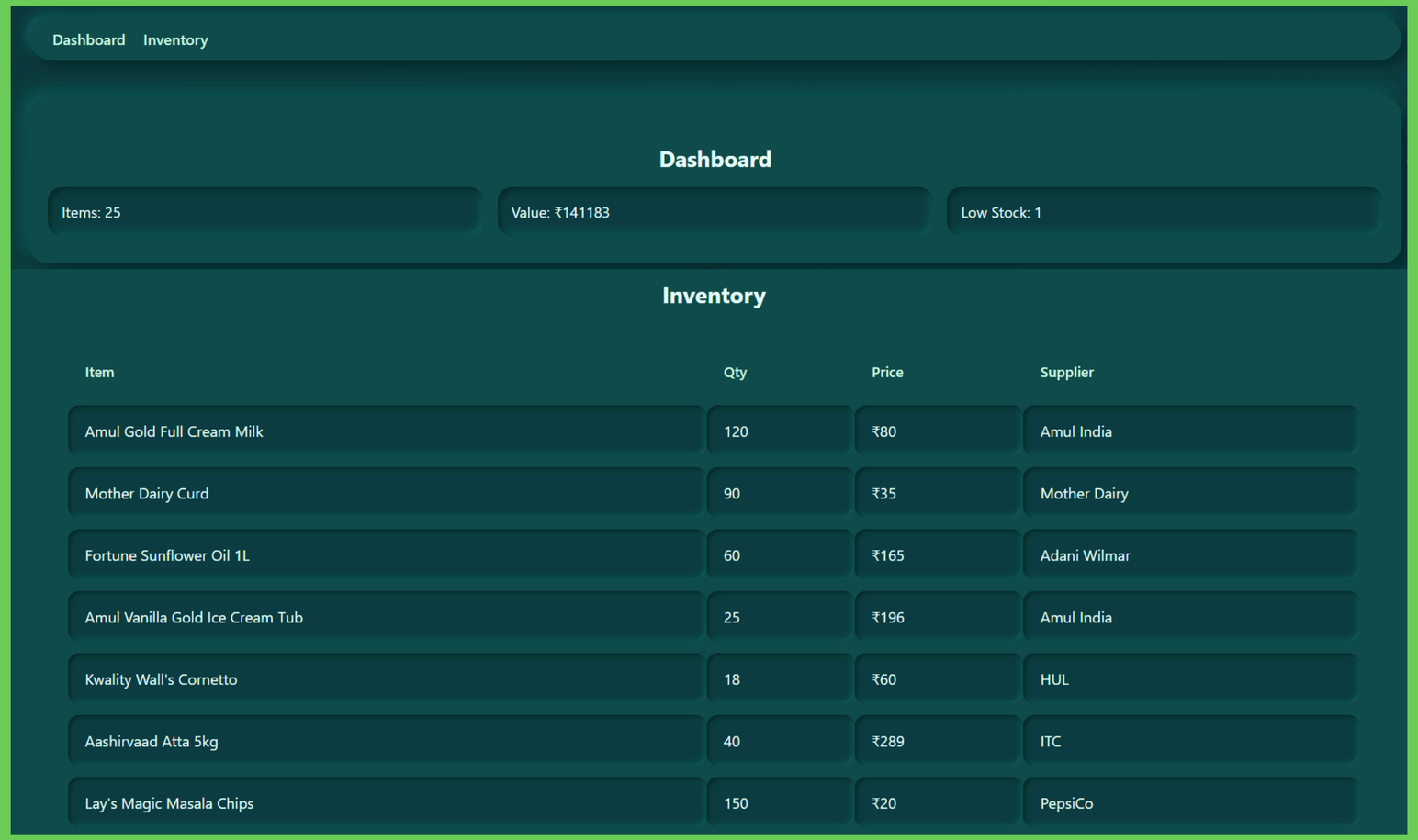
Aggregation for Category Reports

- Category-wise grouping was used to generate stock summaries
- This helps analyze which sections hold the maximum inventory
- Useful for demand and warehouse space planning

```
> db.items.aggregate([
    {
      $group: {
        _id: "$category",
        totalValue: {
          $sum: { $multiply: ["$price", "$quantity"] }
        }
      }
    }
  ])
< {
    _id: 'Personal Care',
    totalValue: 10950
  }
  {
    _id: 'Staples',
    totalValue: 34095
  }
  {
    _id: 'Frozen',
    totalValue: 6700
  }
  {
    _id: 'Dairy',
    totalValue: 15050
  }
```

Frontend Dashboard

- A dashboard interface was developed for warehouse visualization
- It displays stock count, total value, and low stock status
- The inventory section was presented in a structured table layout



Dashboard features

- Dashboard overview
- Stock metrics
- Inventory table
- Low stock alerts
- Total stock value
- Supplier management
- Transaction handling
- Real-time refresh
- Backend synchronization
- Smooth navigation

Backend API Development

#1

Node.js and Express were used to build the backend service

#2

REST endpoints were created for items, suppliers, and transactions

#3

The backend acts as the communication layer between UI and database

Conclusion

The project successfully models an almost real warehouse inventory lifecycle
It combines CRUD, analytics, indexing, and visualization in one system
The design is scalable for larger dark-store warehouse operations

THANK YOU

Unstructured Database
2025-2026